

Кросс-валидация и корректная оценка качества модели

Конспект к практическому занятию

1. Что именно мы оцениваем

Обучая модель, мы хотим найти закономерности, которые сохраняются на новых объектах, а не просто запомнить обучающую таблицу. Способность модели работать на данных, которые она не использовала при обучении, называют обобщающей способностью.

Высокое качество на обучающих данных само по себе ничего не гарантирует. Модель может подстроиться под случайные особенности конкретной выборки — это переобучение. Поэтому для оценки нужны отдельные наблюдения.

При этом оценивать следует весь алгоритм получения предсказаний: заполнение пропусков, масштабирование, отбор признаков и саму модель. Если хотя бы один обучаемый этап использовал информацию из проверочной части, оценка уже не полностью независима.

2. Обучающая, валидационная и тестовая части

Данные разработки и отложенный тест

Данные разработки	Отложенный тест
Обучение и выбор конфигурации через CV	Проверка выбранного решения
Тест не участвует в выборе модели и обучении обработки.	

- Обучающая часть — данные, по которым алгоритм определяет параметры: коэффициенты модели, средние для заполнения пропусков, параметры масштабирования.
- Валидационная часть — данные для сравнения вариантов: моделей, наборов признаков, гиперпараметров и способов обработки.
- Тестовая часть — отложенные данные для финальной проверки уже выбранного решения.

Разница между параметрами и гиперпараметрами принципиальна. Коэффициенты логистической регрессии находятся по обучающим данным. Сила регуляризации и число отбираемых признаков задаются извне и выбираются по валидации.

Если много раз проверять разные решения на одном тесте и выбирать лучшее, тест фактически становится валидацией. Выбор начинает учитывать его случайные особенности, поэтому итоговая метрика может оказаться завышенной.

В ноутбуке используется набор breast cancer для бинарной классификации. Операция $y = 1$ — y меняет обозначения классов: положительным становится класс злокачественных образований. Это важно для интерпретации F1, которая по умолчанию относится к классу 1. На финальный тест отводится 15% данных.

3. Почему одного разбиения может быть мало

Однократное разбиение, или hold-out, просто и недорого: модель обучают на одной части, проверяют на другой. Но результат зависит от состава обеих частей.

- В проверочную часть может случайно попасть непропорционально много лёгких или трудных объектов.
- Для редкого класса может оказаться слишком мало примеров, а иногда класс отсутствует совсем.
- Обучение на уменьшенной выборке может дать более слабую модель, чем обучение на всех доступных данных.

Последний пункт объясняет возможное пессимистичное смещение: измеряется качество алгоритма, обученного на меньшем объёме. Это не то же самое, что недостаточная выразительность модели. Слишком простая модель может недообучаться при любом разбиении.

Hold-out остаётся полезным при большом объёме данных или дорогом обучении. Кросс-валидация уменьшает зависимость от одного выбранного разбиения, но не исправляет нерепрезентативный датасет или ошибочную схему проверки.

4. Стратификация

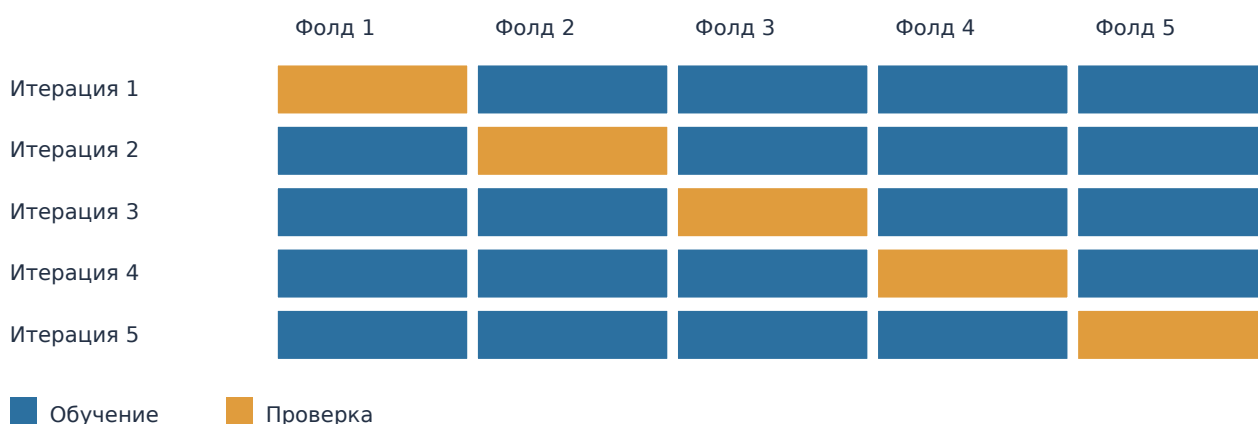
Стратификация — разбиение с приблизительным сохранением долей классов. Если в исходных данных 37% объектов положительного класса, желательно получить близкую долю и в обучающей, и в проверочной части.

В практике это задаётся через `stratify=y`. В сохранённом выводе доля класса 1 составляет примерно 37,27% в train и 37,21% в test. Это иллюстрация сохранения состава, а не искусственного балансирования до 50 на 50.

Для кросс-валидации аналогичный принцип реализует `StratifiedKFold`. Он особенно полезен, когда классы встречаются с разной частотой. Однако стратификация не обеспечивает независимость объектов и не учитывает временной порядок.

5. Как работает K-Fold

K-Fold: проверка на одном фолде



Данные, предназначенные для разработки модели, делят на k непересекающихся частей — фолдов. Далее выполняют k независимых обучений: каждый раз один фолд используют для проверки, остальные — для обучения.

При $k = 5$ схема выглядит так:

- Итерация 1: проверка на фолде 1, обучение на фолдах 2-5.
- Итерация 2: проверка на фолде 2, обучение на фолдах 1, 3-5.
- Аналогично для фолдов 3, 4 и 5.

Каждый объект один раз попадает в проверочную часть. В каждом обучении используется около 80% данных разработки. Финальный тест в эти итерации не входит.

Для значений метрики $s_1, \dots, s_k$ обычно считают среднее: $\text{mean} = (s_1 + \dots + s_k) / k$. Разброс значений помогает увидеть чувствительность к составу выборки.

Важно: стандартное отклонение метрик по фолдам не равно доверительному интервалу среднего. Обучающие части сильно пересекаются, а оценки не являются независимыми повторениями эксперимента. Запись «среднее $\pm$ std» описывает разброс по данным разбиениям, а не гарантированный диапазон качества на новых данных.

Увеличение k повышает долю данных для каждого обучения, но требует больше вычислений и уменьшает проверочные части. Поэтому больше фолдов не означает автоматически более надёжную оценку. Часто используют 5 или 10 фолдов. [\[документация\]](#)

6. KFold, StratifiedKFold и cross_val_score

KFold учитывает позиции строк, но не метки классов. StratifiedKFold дополнительно сохраняет приблизительные пропорции классов. Оба по умолчанию не перемешивают данные.

Перемешивание полезно, когда строки упорядочены по классу или другой несущественной для применения модели величине. Для воспроизводимого случайного разбиения задают `shuffle=True` и целочисленный `random_state`. Для временных данных такое перемешивание обычно не подходит.

В ноутбуке сравниваются автоматический вызов `cross_val_score` и ручные циклы. В ручном варианте на каждой итерации создаётся новая логистическая регрессия. Нельзя продолжать обучение одной модели на следующих фолдах: тогда проверки перестанут быть независимыми от предыдущих обучений.

При вызове `cross_val_score` с классификатором, бинарной целью и `cv=5` scikit-learn по умолчанию использует StratifiedKFold. Поэтому результаты этого вызова совпадают с явным стратифицированным циклом, а не с обычным KFold. Сама `cross_val_score` возвращает оценки, но не одну готовую финальную модель. [\[документация\]](#)

7. Метрики в практике: F1 и ROC-AUC

F1 объединяет точность положительных предсказаний и полноту обнаружения положительного класса:

$$F1 = 2 \cdot TP / (2 \cdot TP + FP + FN).$$

Здесь TP — верные положительные предсказания, FP — ложные положительные, FN — пропущенные положительные объекты. F1 зависит от того, какой класс считается положительным, и от порога принятия решения. Число верных отрицательных предсказаний напрямую в формулу не входит.

ROC-AUC используется в примере с отбором признаков. Она оценивает качество ранжирования объектов по оценке положительного класса при разных порогах. Для расчёта обычно нужны вероятности или значения `decision_function`, а не только метки 0/1.

Эти метрики отвечают на разные вопросы. F1 характеризует решения при конкретном пороге, ROC-AUC — разделение классов по оценкам модели. Их числа нельзя напрямую

сравнивать между собой как «больше — лучше».

8. Утечка данных: почему обработка должна быть внутри CV

Утечка возникает, если при построении модели используется информация, которая не должна быть доступна в соответствующей проверке. Не обязательно передавать в модель правильные ответы напрямую: утечку могут создавать предварительные вычисления.

В практике генерируется 500 объектов с 4000 признаками, из которых только 10 информативны. Затем SelectKBest с критерием `f_classif` оставляет 100 признаков. Критерий сравнивает различия средних значений признака между классами с вариацией внутри классов; каждый признак оценивается отдельно.

Когда признаков очень много, некоторые случайно выглядят связанными с целью. Если отобрать их по всей таблице до CV, в отбор попадут метки будущих проверочных объектов. Модель формально обучится только на train-фолде, но набор её входных признаков уже выбран с помощью validation-фолда.

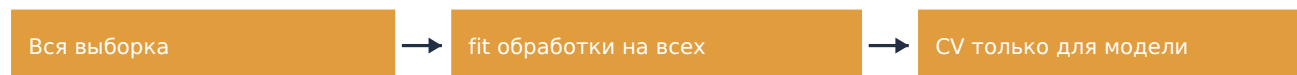
В сохранённом выводе ноутбука:

- Отбор до CV: ROC-AUC = 0,918, стандартное отклонение = 0,026.
- Отбор внутри Pipeline: ROC-AUC = 0,729, стандартное отклонение = 0,026.

Разница показывает влияние утечки в конкретном эксперименте. Эти значения взяты из сохранённых результатов, а не получены повторным запуском; при смене окружения они могут отличаться.

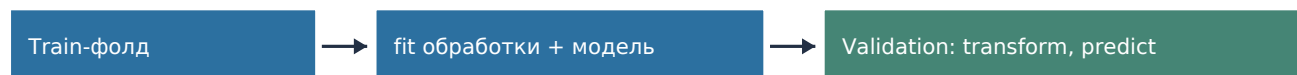
Что обеспечивает Pipeline

Неверно: проверочные данные влияют на обработку



Метки validation могут повлиять на выбор признаков.

Верно: обработка обучается отдельно в каждом фолде



В CV передаётся весь Pipeline, а не заранее обработанная таблица.

Pipeline объединяет последовательность этапов обработки и модель. Если весь Pipeline передать в CV, на каждом фолде он заново обучает преобразования только на обучающей части и применяет их к проверочной части без повторного обучения.

Правильная последовательность для одного фолда:

1. Обучить обработку на train-фолде.
2. Преобразовать train-фолд и обучить модель.
3. Преобразовать validation-фолд с уже найденными параметрами.
4. Получить предсказания и посчитать метрику.

Так нужно поступать с заполнением пропусков по статистикам, масштабированием, отбором признаков, словарём текста и кодированием категорий. Pipeline помогает соблюдать порядок, но не убирает автоматически признаки из будущего и не выбирает

подходящую схему разбиения. [\[документация\]](#)

Уточнение к началу ноутбука

StandardScaler в первых примерах обучается на всём X_{train} до кросс-валидации. Внешний X_{test} не используется, но средние и стандартные отклонения включают будущие validation-фолды. Это утечка внутри CV.

Для корректной оценки StandardScaler следует включить в тот же Pipeline, что и LogisticRegression, и передать в `cross_val_score` исходный X_{train} . Это тот же принцип, который далее демонстрируется на SelectKBest.

9. Что делать после выбора решения

Обычный путь — выбрать конфигурацию по CV и заново обучить весь Pipeline на всех данных разработки. Затем один раз оценить его на отложенном тесте.

Альтернатива из ноутбука — сохранить модели разных фолдов и усреднять их предсказания. В регрессии обычно усредняют числовые ответы, в классификации можно усреднять вероятности и затем применять порог.

Это уже ансамбль — другой алгоритм предсказания. Его качество нужно проверять отдельно: средняя CV-метрика отдельных моделей не является метрикой ансамбля. Также увеличиваются стоимость хранения и время предсказания.

10. Leave-One-Out

Leave-One-Out: проверка на одном объекте

	Объект 1	Объект 2	Объект 3	Объект 4	Объект 5	Объект 6
Итерация 1	Обучение	Обучение	Обучение	Обучение	Обучение	Обучение
Итерация 2	Обучение	Проверка	Обучение	Обучение	Обучение	Обучение
Итерация 3	Обучение	Обучение	Проверка	Обучение	Обучение	Обучение
Итерация 4	Обучение	Обучение	Обучение	Проверка	Обучение	Обучение
Итерация 5	Обучение	Обучение	Обучение	Обучение	Проверка	Обучение
Итерация 6	Обучение	Обучение	Обучение	Обучение	Обучение	Проверка

■ Обучение ■ Проверка

Leave-One-Out — предельный случай K-Fold, в котором k равно числу объектов n . На каждой итерации один объект оставляют для проверки, а на остальных $n - 1$ обучают модель.

Плюс метода — почти все данные участвуют в каждом обучении. Это может быть удобно при очень небольшой выборке. Минус — необходимость обучить n моделей вместо 5 или 10.

Большой объём обучения на каждой итерации не делает оценку автоматически лучшей: полученные модели очень похожи, а отдельные проверки основаны на одном объекте. Оценка ошибки может иметь высокий разброс. [\[документация\]](#)

Почему F1 по одному объекту вводит в заблуждение

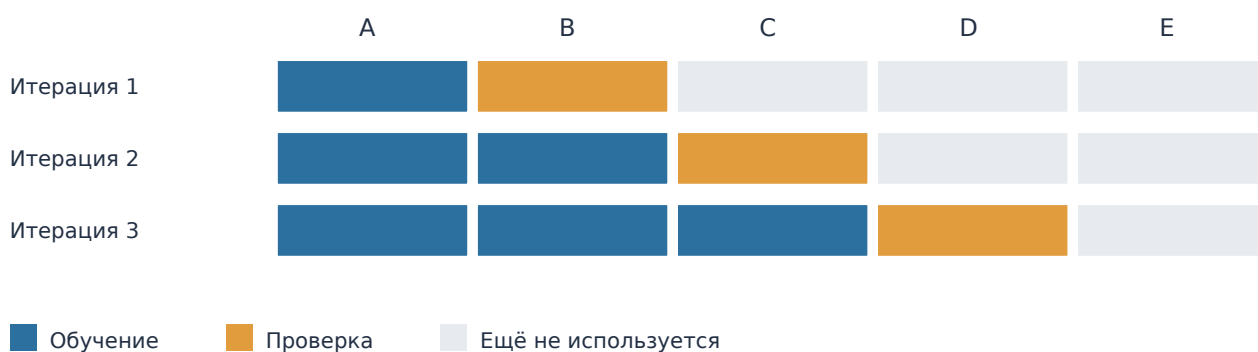
В ноутбуке печатается F1 для отдельных Leave-One-Out-фолдов. При одном проверочном объекте это плохо отражает баланс точности и полноты. Например, если единственный объект отрицательный и верно предсказан как отрицательный, $TP = FP = FN = 0$: при $zero_division=0$ получится $F1 = 0$, хотя ошибки нет.

Для содержательной итоговой F1 можно собрать все предсказания объектов, полученные тогда, когда они были вне обучения, и вычислить F1 по всей совокупности. Это агрегированная метрика по внеобучающим предсказаниям, а не среднее F1 отдельных фолдов.

Не стоит переносить правило агрегирования на любую метрику без проверки: для нелинейных метрик оценка по объединённым предсказаниям и среднее по фолдам вообще могут различаться.

11. Проверка временных рядов

Прошлое → будущее: обучающая история расширяется



Во временных данных соседние наблюдения часто связаны. Если случайно перемешать даты, модель может обучаться на будущем относительно проверочного объекта. Такая проверка не соответствует задаче прогнозирования.

`TimeSeriesSplit` сохраняет порядок: обучение — на более ранних наблюдениях, проверка — на следующем временном участке. В базовом варианте обучающая история постепенно расширяется.

Условная схема:

- Итерация 1: обучение на периоде A, проверка на B.
- Итерация 2: обучение на A + B, проверка на C.
- Итерация 3: обучение на A + B + C, проверка на D.

В практике используется таблица ежедневного потребления электроэнергии Германии. Код показывает разбиения на три временные проверки и рост обучающей части. Он не обучает прогнозную модель и не оценивает качество прогноза.

Сам разбиитель работает с порядком строк, поэтому перед его применением данные нужно упорядочить по времени. При сравнении периодов важно учитывать горизонт прогноза и шаг наблюдений. `gap` позволяет оставить промежуток между обучением и проверкой, если это требуется устройством задачи. Ни один признак не должен использовать события, ещё не известные на момент прогнозирования. [\[документация\]](#)

12. Как выбрать схему проверки

- Независимые объекты, регрессия: KFold с осмысленным перемешиванием.
- Независимые объекты, классификация: обычно StratifiedKFold.
- Очень мало независимых объектов: можно рассмотреть Leave-One-Out с учётом стоимости и метрики.
- Прогноз будущих событий: временное разбиение.
- Несколько строк на одного пациента, пользователя или устройство: нужно учитывать группы, если проверяется перенос на новые группы.

Последний случай — краткое дополнение к теме независимости объектов. Если связанные строки попадают одновременно в обучение и проверку, модель может узнавать объект или его группу вместо обобщения на новые группы. Для такого сценария существуют групповые разбиения. [\[документация\]](#)